Manutenção e Evolução de Software

Programação Estratégica

Universidade do Minho

2025/2026

1 Programação Estratégica em Python

Para o uso de programação estratégica em Python, vai ser usada a biblioteca pyZtrategic, que se baseia na biblioteca Ztrategic com as adaptações necessárias para o funcionamento em Python. Esta biblioteca pode ser encontrada na sua versão mais recente no seguinte repositório: `https://github.com/SLE-Laboratory/pyZtrategic`.

Para o seu uso, a forma mais imediata e eficaz será a instalação de dependências com o uso do Poetry, um dependency manager do Python. Para tal, deverá instalar o Poetry se não o tem instalado no sistema:

```
$ pip install python3-poetry
```

Poderá precisar de atualizar a sua versão do Python para 3.12:

```
$ sudo add-apt-repository ppa:deadsnakes/ppa
$ sudo apt update
$ sudo apt install python3.12
```

Depois deverá fazer download do pyZtrategic presente no Blackboard, seguido do seguinte comando para instalar as dependências necessárias:

```
$ poetry install
```

Com este comando, as dependências ficam instaladas no sistema. Note-se que o Poetry irá sempre usar ambientes virtuais para a manipulação de projetos. Para um uso básico do Poetry, pode-se abrir um ficheiro com esse ambiente virtual com o seguinte comando:

```
$ poetry run python3 -i <ficheiro>
```

Recomenda-se consultar a documentação do Poetry[1] para fazer um uso mais completo desta ferramenta.

Vamos considerar um exemplo muito básico de transformação de uma lista de inteiros. Para o uso da biblioteca *pyZtrategic*, é necessário importá-la.

[1] `http://python-poetry.org/docs/`

```
# Para o uso de Zippers
from pyztrategic.zipper import obj

# Para o uso de programação estratégica
from pyztrategic import strategy as st
```

Vamos agora pegar numa dada lista de inteiros que será recebida como input da função, converter para um zipper, usando a função `obj`, aplicar uma estratégia e voltar a converter para o formato original. Vamos aplicar a estratégia `full_tdTP(step, obj(l))`, em que `step` é o que queremos fazer em cada nodo e `full_tdTP` (full, topdown, type-preserving) transforma essa função para aplicar a toda a estrutura de dados, de uma forma topdown.

```
def alteraDois(i):
    if i == 2:
        return 12
    raise st.StrategicError

def alteraListaS(l):
    def step(x):
        return st.adhocTP(st.idTP, alteraDois, x)
    return st.full_tdTP(step, obj(l)).node()
```

Repare na definição de `step`. Esta função tem por defeito o comportamento `idTP`, que é a função identidade, ao qual juntamos (usando `adhocTP`) a função `alteraDois`. Esta função `alteraDois` será aplicada indiscriminadamente a todos os nodos, e irá alterar todas as ocorrências do número 2 para 12, atirando uma exeção para todos os outros casos. Note que a função `step` pode ser estendida com todos os tipos que forem necessários, podendo assim afetar estruturas de dados heterogéneas. Por exemplo, se se pretender alterar todos os números e todas as strings de uma determinada estrutura de dados, pode-se usar este `step`:

```
def step1(x):
        return st.adhocTP(st.idTP, alteraNumero, x)
def step(x):
        return st.adhocTP(step1, alteraString, x)
```

2 Exercícios

Atente no ficheiro de código que foi disponibilizado na plataforma Blackboard, que contém já implementadas uma estratégia Type-Preserving para a otimização de expressões aritméticas, e uma estratégia Type-Unifying para para contar os números pares de um programa na linguagem `Let`. Note nos exemplos disponibilizados que esta biblioteca também está construída sobre Zippers, sendo que a função `obj(<...>)` corresponde à função `toZipper` e o método `<...>.node()` corresponde à função `fromZipper`.

1. Descarregue o zip do pyZtrategic e o ficheiro de código disponibilizado. Abra o ficheiro disponibilizado, e analise o resultado de invocar `print(ex1)`, e de invocar `print(optimise(f1))`. Analise os resultados obtidos.

2. Defina uma estratégia que percorra um programa `Let` e etiquete todas as variáveis com o prefixo "v_".

3. Expanda agora a estratégia `optimise` de maneira a que simplifique também todas as ocorrências de $x * 1$ para x, sendo que x representa qualquer expressão aritmética.

4. Finalmente, expanda novamente a estratégia `optimise` para simplificar todas as expressões aritméticas entre valores `Const`, i.e, `Add(Const(3), Const(4))` para `Const(7)`, `Mul(Const(3), Const(4))` para `Const(12)`, etc.

5. Defina uma nova estratégia Type-Unifying para recolher todos os nomes de variáveis que ocorrem num programa na linguagem `Let`.